Project Report

1. Introduction

Process management is a fundamental concept in operating systems that enables a system to run multiple programs concurrently. In Unix-based systems, processes are created, executed, and terminated using well-defined system calls such as fork(), exec(), and wait(). Understanding how these calls interact is essential for learning how operating systems manage multitasking, resource allocation, and program execution.

This project focuses on basic Unix process management by implementing a C program that creates multiple child processes, replaces their execution images with external commands, and synchronizes their termination using the parent process. The goal is to understand process creation order, execution replacement, termination behavior, and how exit statuses and signals are handled.

2. Implementation Summary

The program consists of a single parent process that creates 15 child processes using a loop and the fork() system call. The parent process prints its own process ID (PID) at the beginning of execution and stores the PID of each child in an array in the order they are created.

Each child process performs the following steps:

1. Prints its child index, PID, and the command it will execute.
2. Executes a unique Linux command using execvp().

Several special cases are included to demonstrate different termination behaviors:

- One child executes echo "Hello Esther Law" to demonstrate successful execution.
- Two child processes intentionally execute invalid commands, causing execvp() to fail and the child to exit with a non-zero exit code (127).
- Two child processes intentionally terminate abnormally using the abort() function, demonstrating termination by a signal.

The parent process waits for each child to terminate using waitpid() in the same order the children were created. For each child, the parent determines whether the process exited normally or was terminated by a signal and reports the appropriate exit code or signal number.

3. Results and Observations

Child Process Creation and PID Management

All child processes are created inside a loop using fork(). The return value of fork() is used to distinguish between parent and child processes. The parent stores each child’s PID in an array

immediately after creation, ensuring that the PIDs are recorded in creation order rather than completion order.

Each child prints identifying information before executing its assigned command, making it easy to observe the behavior of individual processes during execution.

Process Creation Order vs. Termination Order

Although child processes are created sequentially, they do not necessarily terminate in the same order. Some commands complete quickly, while others (such as those terminated by abort() or delayed by system execution timing) may finish later.

To clearly demonstrate the difference between creation order and completion order, the parent process uses waitpid() with specific child PIDs. This forces the parent to wait for each child in the exact order they were created, regardless of when they actually terminate.

Exit Status and Signal Handling

The parent process uses macros provided by <sys/wait.h> to interpret each child's termination status:

- WIFEXITED(status) is used to detect normal termination.
- WEXITSTATUS(status) retrieves the exit code for normally exited processes.
- WIFSIGNALED(status) determines if a process was terminated by a signal.
- WTERMSIG(status) identifies the signal number responsible for termination.

The program maintains counters for:

- Processes that exited normally with exit code 0
- Processes that exited normally with a non-zero exit code
- Processes that were terminated by a signal

These counts are displayed in a final summary after all child processes have completed.

4. Conclusion

This project provided hands-on experience with Unix process management and reinforced the roles of fork(), execvp(), and waitpid() in process control. By implementing multiple child processes with varying execution outcomes, the project demonstrated how processes are created, how execution can be replaced, and how termination status is communicated back to the parent.

The distinction between process creation order and termination order became clear through the use of waitpid() with specific PIDs. Additionally, handling both normal exits and signal-based

termination improved understanding of how operating systems manage and report process lifecycles.

Overall, this project strengthened practical knowledge of Unix process management and provided a solid foundation for more advanced topics such as inter-process communication and scheduling.